

BNF and Context-Free Grammars

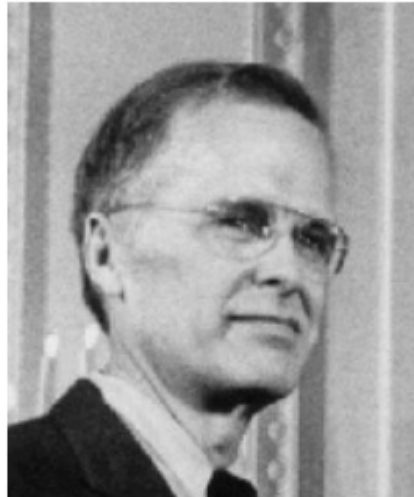
- Context-Free Grammars
 - Developed by Noam Chomsky in the mid-1950s
 - Language generators, meant to describe the syntax of natural languages
 - Define a class of languages called context-free languages
- Backus-Naur Form (1959)
 - Invented by John Backus to describe the syntax of Algol 58
 - BNF is equivalent to context-free grammars

What is BNF?

- It stands for Backus-Naur Form
- It is a formal, mathematical way to specify context-free grammars
- It is precise and unambiguous
- Before BNF, people specified programming languages ambiguously, i.e., with English

- John Backus presented a new notation containing most of the elements of BNF at a UNESCO conference
- His presentation was about Algol 58
- Peter Naur read this report and found that he and Backus interpreted Algol differently
- He wanted even more precision
- So he created what we now know as BNF for Algol 60
- Thus BNF was first published in *Algol 60 Report*

Who was John Backus?



- Backus invented FORTRAN (“FORMula TRANslator”), the first high-level language *ever*, circa 1954
- Major influence on the invention of functional programming in 1970’s
- Won the 1977 Turing Award for BNF and FORTRAN

Who was Peter Naur?



- Danish astronomer turned computer scientist
- Born in 1928; picture on left is from 1968

BNF Fundamentals

- In BNF, abstractions are used to represent classes of syntactic structures—they act like syntactic variables (also called *nonterminal symbols*, or just *terminals*)
- *Terminals* are lexemes or tokens
- A rule has a left-hand side (LHS), which is a nonterminal, and a right-hand side (RHS), which is a string of terminals and/or nonterminals

What does BNF look like?

- Like this:

`<number> ::= <digit> | <number> <digit>`

`<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

- “::=” means “is defined as” (some variants use “:=” instead)
- “|” means “or”
- Angle brackets mean a nonterminal
- Symbols without angle brackets are terminals

More BNF Examples

- `<while loop> ::= while (<condition>)
 <statement>`
- `<assignment statement> ::= <variable> =
 <expression>`
- `<statement list> ::= <statement> |
 <statement list> <statement>`
- `<unsigned integer> ::= <digit> |
 <unsigned integer><digit>`

BNF for Expressions

`<expression> ::= <expression> + <term>`
`| <expression> - <term>`
`| <term>`

`<term> ::= <term> * <factor>`
`| <term> / <factor>`
`| <factor>`

`<factor> ::= <primary> ^ <factor>`
`| <primary>`

`<primary> ::= <primary>`
`| <element>`

`<element> ::= ( <expression> )`
`| <variable>`
`| <number>`

BNF Fundamentals (continued)

- Nonterminals are often enclosed in angle brackets
 - Examples of BNF rules:
`<ident_list> → identifier | identifier, <ident_list>`
`<if_stmt> → if <logic_expr> then <stmt>`
- Grammar: a finite non-empty set of rules
- A *start symbol* is a special element of the nonterminals of a grammar

BNF Rules

- An abstraction (or nonterminal symbol) can have more than one RHS

```
<stmt> → <single_stmt>  
        | begin <stmt_list> end
```

What is EBNF?

- EBNF is a few simple extensions to BNF which make expressing grammars more convenient; adds “syntactic sugar”
- Thus it is more concise than BNF
- EBNF is no more “powerful” than BNF; that is, anything that can be expressed in EBNF can also be expressed in BNF
- EBNF is widely used as the de facto standard to define programming languages

What are the Extensions?

- They vary, but often are derived from regular expression syntax
- “*” (The Kleene Star): means 0 or more occurrences
- “+” (The Kleene Cross): means 1 or more occurrences
- “?”: means 0 or 1 occurrences (sometimes “[...]” used instead)
- Use of parentheses for grouping

BNF vs EBNF

- Grammar for decimal numbers in plain BNF:

`<expr> ::= '-' <num> | <num>`

`<num> ::= <digits>`

`| <digits> '.' <digits>`

`<digits> ::= <digit>`

`| <digit> <digits>`

`<digit> ::= '0' | '1' | '2' | '3' |
 '4' | '5' | '6' | '7' | '8' | '9'`

BNF vs EBNF

- Same grammar in EBNF:

```
<expr>  := '-'? <digit>+ ('.' <digit>+)?  
<digit> := '0' | '1' | '2' | '3' | '4' |  
          '5' | '6' | '7' | '8' | '9'
```

- So much more concise!
- An optional '-', one or more digits, an optional decimal point followed by one or more digits

Simple Conversions

- If you have a rule such as:

`<expr> ::= <digits>`

`<digits> ::= <digit>`

`| <digit> <digits>`

- You can replace it with:

`<expr> ::= <digit>+`

More Simple Conversions

- If you have a rule such as:

`<expr> ::= <digits> | empty`

- You can replace it with:

`<expr> ::= <digit>*`

More Simple Conversions

- If you have a rule such as:

```
<id> ::= <letter>  
        | <id><letter>  
        | <id><digit>
```

- You can replace it with:

```
<id> ::= <letter> (<letter> | <digit>)*
```

EBNF for Lisp

```
s_expression ::= atomic_symbol  
               | "(" s_expression "." s_expression ")"  
               | list  
list ::= "(" s_expression* ")"  
atomic_symbol ::= letter atom_part  
atom_part ::= empty  
              | letter atom_part  
              | number atom_part  
letter ::= "a" | "b" | " ..." | "z"  
number ::= "1" | "2" | " ..." | "9"
```

Formalizing Description – EBNF

- An EBNF description is an unordered list of EBNF rules.
- Each EBNF rule has three parts:
 - a left-hand side (LHS),
 - LHS is one italicized word (possibly with underscores) written in lower-case; it names the EBNF rule.
 - a right-hand side (RHS),
 - The RHS supplies a description of this name
 - It can include combinations of the four control forms
 - The $\Leftarrow$ character separating these two sides

Table 1.1: Control Forms of Right-Hand Sides

Sequence	Items appear left-to-right; their order is important.
Choice	Alternative items are separated by a (stroke); one item is chosen from this list of alternatives; their order is unimportant.
Option	The optional item is enclosed between [and] (square-brackets); the item can be either included or discarded.
Repetition	The repeatable item is enclosed between { and } (curly-braces); the item can be repeated zero or more times; yes, we can choose to repeat zero times, fact beginners often forget.

EBNF Special Characters

$\Leftarrow$, “is defined as”

|,

[,

],

{, and

}

An EBNF Description of Integers

The following EBNF rules describe how to write simple integers.¹ Their RHS illustrates every control form available in EBNF. An EBNF description: *integer*

EBNF Description: *integer*

digit $\Leftarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

integer $\Leftarrow$ [+|-]*digit*{*digit*}

We can paraphrase the meanings of these rules in English.

- A *digit* is defined as one of the ten alternative characters 0 through 9.
- An *integer* is defined as a sequence of three items: an optional sign (if it is included, it must be one of the alternatives + or -), followed by any *digit*, followed by a repetition of zero or more *digits*, where each repeated digit is independently chosen from the list of alternatives in the *digit* rule.

What do these EBNF rules mean? We can paraphrase their meaning in English

- The RHS of *integer* combines all the control forms in EBNF: sequence, option, choice, and repetition.

Equivalent EBNF Descriptions

EBNF Description: *integer* (equivalent, in 3 rules)

sign $\Leftarrow +|-$

digit $\Leftarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

integer $\Leftarrow [sign]digit\{digit\}$

This EBNF description is not identical to the first, because it defines an extra *sign* rule that is then used in the *integer* rule. But these two EBNF descriptions are equivalent, because providing a named rule for $[+|-]$ does not change which symbols are legal. In fact, even if the names of all the rules are changed uniformly, exactly the same symbols are recognized as legal.

Extra names do not change the meanings of EBNF descriptions; nor do different names for the rules

EBNF Description: *z* (really *integer* with different rule names)

x $\Leftarrow +|-$

y $\Leftarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

z $\Leftarrow [x]y\{y\}$

EBNF Description: *integer_set*

integer_list $\Leftarrow integer\{,integer\}$

integer_set $\Leftarrow \boxed{\{[integer_list]\}}$

Describing Lists

- Syntactic lists are described using recursion

$$\begin{aligned} \text{<ident_list>} &\rightarrow \text{ident} \\ &\quad | \text{ ident, <ident_list>} \end{aligned}$$

- A derivation is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)

An Example Grammar

$\langle \text{program} \rangle \rightarrow \langle \text{stmts} \rangle$

$\langle \text{stmts} \rangle \rightarrow \langle \text{stmt} \rangle \mid \langle \text{stmt} \rangle ; \langle \text{stmts} \rangle$

$\langle \text{stmt} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$

$\langle \text{var} \rangle \rightarrow a \mid b \mid c \mid d$

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle - \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{var} \rangle \mid \text{const}$

An Example Derivation

$\langle \text{program} \rangle \Rightarrow \langle \text{stmts} \rangle \Rightarrow \langle \text{stmt} \rangle$
 $\Rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$
 $\Rightarrow a = \langle \text{expr} \rangle$
 $\Rightarrow a = \langle \text{term} \rangle + \langle \text{term} \rangle$
 $\Rightarrow a = \langle \text{var} \rangle + \langle \text{term} \rangle$
 $\Rightarrow a = b + \langle \text{term} \rangle$
 $\Rightarrow a = b + \text{const}$

Derivations

- Every string of symbols in a derivation is a *sentential form*
- A *sentence* is a sentential form that has only terminal symbols
- A *leftmost derivation* is one in which the leftmost nonterminal in each sentential form is the one that is expanded
- A derivation may be neither leftmost nor rightmost